

Esame Software Engineering (AA 2025/26)

10 Aprile 2026 - Lab. Colossus - Via salaria 113

Enrico Tronci

*Computer Science Department, Sapienza University of Rome
Via Salaria 113 - 00198 Roma - Italy*

tronci@di.uniroma1.it

<https://raise.uniroma1.it>

Esercizio 3 (20 punti)

Un sistema di *e-commerce* ha C customers, S servers, F fornitori ed un DB centralizzato. I customer sono numerati $1, 2, \dots, C$. I server sono numerati $1, 2, \dots, S$. I fornitori sono numerati $1, 2, \dots, F$.

Ciascun *customers* può essere modellato come una catena di Markov con un solo stato (0). Ciascun customer resta nello stato 0 per un tempo τ scelto uniformemente a random nell'intervallo $[A, B]$.

Se il customer decide di fare una transizione (che lo riporta nello stato 0) allora:

1. Sceglie l'indice $s \in \{1, \dots, S\}$ del server come segue. Con probabilità S_0 sceglie la classe di servers con indice pari, e con probabilità $(1 - S_0)$ sceglie la classe di servers con indice dispari. All'interno della classe di indici scelta (pari o dispari), si sceglie s uniformemente a random.
2. Sceglie l'indice $i \in \{1, \dots, P\}$ del prodotto come segue. Con probabilità P_0 sceglie la classe di prodotti con indice pari, e con probabilità $(1 - P_0)$ sceglie la classe di prodotti con indice dispari. All'interno della classe di indici scelta (pari o dispari), si sceglie i uniformemente a random.
3. Sceglie la quantità $q \in \{1, \dots, Q\}$ del prodotto come segue. Con probabilità Q_0 sceglie la classe di prodotti con indice pari, e con probabilità $(1 - Q_0)$ sceglie la classe di prodotti con indice dispari. All'interno della classe di indici scelta (pari o dispari), si sceglie q uniformemente a random.

Questo vuol dire che il customer ha richiesto al server s , q items del prodotto i . Si noti che quando il customer permane nello stato 0 nulla viene inviato ai servers.

Se sono presenti solo indici dispari (e.g., $S = 1$), allora deve essere 0 la probabilità di selezionare un indice pari (e.g., $S_0 = 0$).

Il customer invia la server s la richiesta (i, q) ed attende dal server la risposta (i, k, v) che definisce quanti items (k) del prodotto i vengono consegnati ed il loro prezzo unitario, cioè per item (v).

I servers sono tutti uguali. Ciascun server ha una variabile `SellBuy` dove memorizza il reward ottenuto dalle vendite ai customers ed il costo sostenuto per acquistare i prodotti dal DB.

La variabile **SellBuy** è inizializzata a 0. Ciascun server ha inoltre una cache locale dove sono memorizzate triple del tipo (i, q, v) dove: i è l'indice di un prodotto, q la quantità disponibile del prodotto i , v il costo unitario del prodotto i .

Quando un server riceve una richiesta (i, q) dal customer j esegue le seguenti operazioni:

- Verifica con la sua cache locale il numero di items disponibili per il prodotto i . Sia (i, g, v) il contenuto della sua cache locale per il prodotto i e sia $k = \min(q, g)$. Il server decrementa di k il numero di items disponibili per il prodotto i , incrementa di kv la sua variabile **SellBuy** di reward e risponde al customer con la coppia (i, k, v) .

Ogni τ_S secondi il server aggiorna la sua cache locale. Il valore τ_S vale **ST0** per i servers di indice pari ed **ST1** per i servers di indice dispari.

La cache locale viene aggiornata come segue.

- Il server invia al DB una richiesta (i, q) . Il DB verifica il numero di items disponibili per il prodotto i . Sia g tale numero e sia $k = \min(q, g)$. Il DB decrementa di k il numero di items disponibili per il prodotto i risponde al server con la coppia (i, k, v) . Il server a sua volta incrementa di k il numero di items disponibili per il prodotto i e decrementa di kv la sua variabile di reward **SellBuy** (poichè ha *pagato* il DB per gli items richiesti). I server di indice pari [dispari] scelgono con probabilità *SEP* [*SOP*] un valore pari [dispari] per i , e con probabilità *QEA* [*QOA*] un valore pari [dispari] per q . All'interno delle classi (pari o dispari), la scelta viene fatta uniformemente a random.

Ciascun server s ha una variabile $y(s)$ che tiene conto dei costi di stoccaggio. Al tempo 0, $y(s)$ vale 0, cioè $y(s, 0) = 0$. Se nella cache locale del server s è presente la tripla (i, q, v) , allora il costo di stoccaggio per s è $\alpha(s, i) = qv$.

Ogni T secondi il server s aggiorna la variabile $y(s)$ come segue.

$$y(s, t + T) = y(s, t) + T0.1 \sum_{i=1}^P \alpha(s, i) \quad (1)$$

(2)

Sia $\beta(s, t)$ il valore al tempo t della variabile **SellBuy** per il server s . Il *reward rate* $RR(t)$ al tempo t è definito come segue.

$$RR(t) = \sum_{s=1}^S \frac{\beta(s, t) - y(s, t)}{t} \quad (3)$$

(4)

Il reward rate misura la profittabilità del sistema.

Ciascun *fornitore* può essere modellato come una catena di Markov con un solo stato (0). Ciascun fornitore resta nello stato 0 per un tempo γ scelto uniformemente a random nell'intervallo $[V, W]$.

Se il fornitore decide di fare una transizione (che lo riporta nello stato 0) allora invia al DB centralizzato il messaggio (i, q) dove

1. il prodotto i è scelto a random tra P prodotti numerati $1, 2, \dots, P$;
2. la quantità q del prodotto i è scelta a random in $\{1, 2, \dots, Q\}$;

Il DB centralizzato ha due tabelle. Una che contiene le coppie (i, v) che definisce il costo unitario degli items del prodotto $i \in \{1, 2, \dots, P\}$.

Il DB centralizzato riceve prodotti dai fornitori e richieste dai servers.

Per ognuno dei P prodotti, il DB è inizializzato a random con valori compresi tra 0 e Q per la quantità e con un intero a random tra 1 e 100 per il costo.

Quando il DB riceve un messaggio (i, q) da un fornitore incrementa di q il numero di items disponibili per il prodotto i ed invia un messaggio di ack al fornitore.

Quando il DB riceve una richiesta dal server si comporta come descritto sopra.

Tutte gli scambi di messaggi impiegano tempo r .

1 Obiettivo

Si vuole riportare in uscita il valore atteso del *reward rate* alla fine della simulazione. Cioè il valore medio del *reward rate* alla fine della simulazione calcolato usando M simulazioni Montecarlo.

2 Parametri

I parametri della simulazione sono:

1. H: orizzonte di simulazione (valore reale positivo)
2. M: numero di simulazioni montecarlo (valore intero positivo)
3. C: numero di customers (valore intero positivo)
4. S: numero di servers (valore intero positivo)
5. F: numero di fornitori (valore intero positivo)
6. A: parametro A del testo (valore reale positivo)
7. B: parametro B del testo (valore reale positivo)
8. V: parametro V del testo (valore reale positivo)

9. W: parametro W del testo (valore reale positivo)
10. P: numeri prodotti (valore intero positivo)
11. Q: parametro Q nel testo (valore intero positivo)
12. r: tempo scambio messaggi (valore reale positivo)
13. S0: probabilità (valore reale in $[0, 1]$)
14. P0: probabilità (valore reale in $[0, 1]$)
15. Q0: probabilità (valore reale in $[0, 1]$)
16. ST0: tempo update server indice pari (valore reale positivo)
17. ST1: tempo update server indice dispari (valore reale positivo)
18. SEP: probabilità (valore reale in $[0, 1]$)
19. SOP: probabilità (valore reale in $[0, 1]$)
20. QEA: probabilità (valore reale in $[0, 1]$)
21. QOA: probabilità (valore reale in $[0, 1]$)

3 Formato dei parametri di input

I parametri della simulazione sono forniti nel file `parameters.txt`. Le righe del file `parameters.txt` hanno tutte lo stesso formato:

<ID del parametro> <valore del parametro>

Un esempio di file `parameters.txt` è:

```
H 234.6
M 100
C 10
A 1.0
B 2.0
Q 8
V 3.0
W 5.0
P 10
S 4
F 2
r 0.01
S0 0.3
P0 0.4
Q0 0.6
ST0 5.1
```

ST1 7
SEP 0.1
SOP 0.4
QEA 0.3
QOA 0.7

4 Formato di output

L'output dell'esercizio è memorizzato nel file `results.txt` la cui prima riga è formattata come indicato nelle istruzioni generali.

Le rimanenti righe del file `results.txt` hanno il formato

$RR < \text{valore medio reward rate a fine simulazione} >$

Un esempio di file `results.txt` per il file `parameters.txt` dato sopra è:

2025-01-09-Mario-Rossi-1234567
RR 12.3

5 Struttura Sorgenti

Deve esserci almeno una classe, in un proprio file `cpp`, per ognuno dei processi: `customer`, `server`, `supplier`, `db`. Quindi devono esserci almeno i seguenti files: `customer.cpp`, `server.cpp`, `supplier.cpp`, `db.cpp`.